Лабораторная работа №11. Оптимизация образов и безопасность Docker

Тема: Multi-stage builds, USER, ограничения ресурсов, read-only filesystem и аудит образов

Цель: Собрать и сравнить два образа Express API, настроить безопасный запуск в Compose и проверить работоспособность системы без потери данных PostgreSQL.

Дисциплина: Docker и контейнеризация приложений

---

1. Общие сведения

Лабораторная работа №11. Оптимизация образов, безопасность и подготовка к Production. Продолжительность: 4 академических часа (180 минут).

Цель: научиться сравнивать образы на реальных измерениях, применять минимальные привилегии и ограничения ресурсов, оценивать результаты анализа уязвимостей.

2. Предварительные условия

- Debian 13, Docker Engine и Docker Compose V2, доступ к терминалу (команды при необходимости выполняются с sudo).
- Рабочая копия проекта лабораторной №9–10: сервисы nginx, api и db, маршрут GET /api/products.
- Каталог api содержит app.js, package.json, package-lock.json, маршруты, контроллеры, сервисы и конфигурацию подключения к db.
- Применяйте изменения к отдельной копии проекта compose-course11; не используйте docker compose down -v.
- Для задания по сканированию Docker Scout должен быть доступен; если его нет, используйте демонстрацию преподавателя.

Перед копированием убедитесь, что в проекте из ЛР №9 нет реальных паролей, токенов и персональных данных. Не прикладывайте значения секретов в отчёт.

3. Планируемые результаты

- Создать Dockerfile.baseline и Dockerfile.production для одного Express API.
- Сравнить фактические размеры и содержимое образов.
- Объяснить назначение multi-stage build, npm ci --omit=dev и .dockerignore.
- Запустить API от пользователя node, применить mem_limit, cpus, read_only и tmpfs.
- Проверить REST API через Nginx, обеспечить сохранность данных PostgreSQL.
- При наличии Docker Scout выполнить анализ двух образов и корректно интерпретировать результат.

4. Подготовка безопасной копии проекта

Скопируйте проект из ЛР №9–10 в новый каталог compose-course11. Все команды ниже выполняются из корня нового проекта, если отдельно не указано иное. Подключённый том БД сохраните отдельно от старого проекта; не запускайте два контейнера PostgreSQL одновременно на одном каталоге данных.

Проверить исходные сервисы:

```
docker compose config -q
docker compose ps
curl -i http://127.0.0.1:8080/api/products
```

Если в вашей .env используется другой порт, подставьте его. Зафиксируйте успешный HTTP-ответ перед изменениями.

5. Эксперимент №1. Исходный образ (baseline)

Создайте файл api/Dockerfile.baseline. Это учебный вариант без оптимизации, он будет точкой сравнения.

Содержимое api/Dockerfile.baseline:

```
FROM node:22
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "app.js"]
```

Сборка baseline:

```
docker build -f api/Dockerfile.baseline -t course-api:baseline ./api
docker image ls course-api:baseline
docker history course-api:baseline
```

Запишите фактический размер образа. Нельзя подставлять заранее известную цифру: значения зависят от версии образа и зависимостей.

6. Эксперимент №2. Оптимизированный многоэтапный образ

Создайте файл api/.dockerignore. Он запрещает перенос локальных зависимостей, Git-истории и файлов окружения в контекст сборки.

Содержимое api/.dockerignore:

```
node_modules
.git
.env
.env.*
!.env.example
*.log
coverage
Dockerfile.baseline
Dockerfile.production
```

Создайте файл api/Dockerfile.production. Для учебного проекта без этапа компиляции первый этап устанавливает production-зависимости, второй формирует runtime-образ.

Содержимое api/Dockerfile.production:

```
FROM node:22-bookworm-slim AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev

FROM node:22-bookworm-slim AS runtime
ENV NODE_ENV=production
```

```
WORKDIR /app
COPY --from=deps --chown=node:node /app/node_modules ./node_modules
COPY --chown=node:node . .
USER node
EXPOSE 3000
CMD ["node", "app.js"]
```

Если npm ci сообщает об ошибке, проверьте наличие package-lock.json и его соответствие package.json. Не заменяйте npm ci на npm install без выяснения причины.

Собрать второй образ и измерить:

```
docker build -f api/Dockerfile.production -t course-api:prod ./api
docker image ls course-api
docker history course-api:prod
```

Сравнение двух образов

Показатель	baseline	production
Базовый образ	node:22	node:22-bookworm-slim
Размер (МБ)	Вписать измеренное	Вписать измеренное
UID процесса	Вписать результат	Вписать результат
Количество слоёв / история	Вписать	Вписать
Работа GET /api/products	Проверить	Проверить

Рассчитайте процент уменьшения: (размер baseline − размер production) / размер baseline × 100%. Если число получилось отрицательным, образ не уменьшился; это допустимый результат эксперимента.

7. Проверка пользователя и прав

Проверить UID:

```
docker run --rm --entrypoint id course-api:baseline
docker run --rm --entrypoint id course-api:prod
docker image inspect course-api:prod --format "{{.Config.User}}"
```

Для baseline в типичной конфигурации ожидается UID 0, для production — UID пользователя node (проверить фактическим выводом).

Проверить запрет записи в корень контейнера:

```
docker run --rm --read-only --entrypoint sh course-api:prod -c "touch /app/test"
```

Ожидается отказ записи в /app. Это диагностический эксперимент: код завершения ненулевой, поэтому не рассматривайте его как поломку образа.

8. Применение production-образа в Docker Compose

Измените только сервис api в compose.yaml, сохраняя исходные environment/env_file, depends_on и настройки сети. Приведённый ниже фрагмент следует объединить с существующим описанием сервиса, а не заменять им весь проект.

Фрагмент для api (вставить в существующий services.api):

```
build:
  context: ./api
  dockerfile: Dockerfile.production
mem_limit: 256m
cpus: 0.50
read_only: true
tmpfs:
  - /tmp
security_opt:
  - no-new-privileges:true
```

ВНИМАНИЕ: код отступает относительно ключа api в compose.yaml. Применяйте CPU/RAM/readonly только к API, не к PostgreSQL.

Проверить и пересоздать только API:

```
docker compose config -q
docker compose up -d --build --no-deps api
docker compose ps
docker compose logs --tail=30 api
```

Проверить, что API по-прежнему работает:

```
curl -i http://127.0.0.1:8080/api/products
docker compose exec api id
docker stats --no-stream
```

Если API перестал работать, изучите логи и права на файлы. При необходимости временно уберите read_only, выясните, куда пишет приложение, и добавьте отдельный tmpfs или volume. Не отключайте права безопасности без объяснения причины.

9. Проверка сохранности данных

Через GET /api/products убедитесь, что ранее добавленные товары доступны. Пересоздание api не должно приводить к удалению тома db. Команду docker compose down -v не использовать.

10. Проверка Docker Scout (если установлен)

Проверить наличие инструмента и выполнить краткий анализ:

```
docker scout version
docker scout quickview course-api:baseline
docker scout quickview course-api:prod
```

Если Scout CLI отсутствует либо требует авторизации, отразите это в отчёте и проанализируйте результаты демонстрации преподавателя. Не утверждайте, что образ безопасен только потому, что сканер не сообщил CVE.

Отчёт сканирования

Показатель	baseline	production
Дата и инструмент	Вписать	Вписать
Critical CVE	Вписать/нет отчёта	Вписать/нет отчёта
High CVE	Вписать/нет отчёта	Вписать/нет отчёта
Доступны исправления	Указать	Указать
Комментарий по рискам	Обоснование	Обоснование

11. Самостоятельное задание

- Объясните, какие файлы были исключены из контекста с помощью .dockerignore и почему.
- Добавьте в compose.yaml ограничение pids_limit для api (например, 100), затем подтвердите значение в итоговой конфигурации.
- Укажите, какие изменения дали реальную экономию места, а какие усилили безопасность без изменения размера.
- Докажите работоспособность GET /api/products после пересоздания api.

12. Требования к отчёту

- Тема, цель и конфигурация стенда.
- Листинги Dockerfile.baseline, Dockerfile.production и .dockerignore.
- Таблица реальных размеров и расчёт процента изменения.
- Вывод команд id, docker image ls и docker compose ps.
- Фрагмент конфигурации с mem_limit, cpus, read_only, tmpfs и security_opt.
- Ответ GET /api/products после пересоздания api.
- Результаты Docker Scout либо объяснение недоступности инструмента и разбор демонстрации.
- Вывод о компромиссах размера, безопасности и совместимости.

13. Контрольные вопросы

- Чем опасен root внутри контейнера?
- Что меняет USER node?
- Что означает --read-only и для чего нужен tmpfs?
- Какие задачи решает multi-stage build?
- Почему важно наличие package-lock.json для npm ci?
- Как .dockerignore влияет на контекст сборки?
- Всегда ли multi-stage уменьшает образ?
- Почему количество CVE не является окончательным показателем безопасности?
- Зачем применять mem_limit и cpus?
- Почему нельзя выполнять docker compose down -v на учебной БД?

14. Критерии оценивания

Оценивание (100 баллов)

Критерий	Баллы
Две корректные сборки и измерения	25
Multi-stage и .dockerignore	20
USER node и проверка прав	15
Ограничения Compose и read_only	15
Проверка REST API и сохранности БД	10
Анализ Scout или результатов демонстрации	5
Отчёт и контрольные вопросы	10
Итого	100

15. Источники и техника безопасности

- Dockerfile reference: https://docs.docker.com/reference/dockerfile/
- Multi-stage builds: https://docs.docker.com/build/building/multi-stage/
- Build best practices: https://docs.docker.com/build/building/best-practices/
- Compose services: https://docs.docker.com/reference/compose-file/services/
- Docker Scout quickview: https://docs.docker.com/reference/cli/docker/scout/quickview/

Во всех экспериментах используйте тестовые данные и отдельную рабочую копию. Не используйте --privileged и не удаляйте том PostgreSQL. Конкретные результаты измерений должны быть получены на вашей учебной машине.